

Modern Operating Systems — Chapter 1 Exercises and Answers

Andrew S. Tanenbaum and Herbert Bos

Contents

Chapter 1	3
Exercise 1	3
Exercise 2	3
Exercise 3	4
Exercise 4	4
Exercise 5	4
Exercise 6	5
Exercise 7	5
Exercise 8	6
Exercise 9	6
Exercise 10	6
Exercise 11	7
Exercise 12	7
Exercise 13	8
Exercise 14	8
Exercise 15	9

Exercise 16	9
Exercise 17	10
Exercise 18	10
Exercise 19	10
Exercise 20	11
Exercise 21	11
Exercise 22	11
Exercise 23	12
Exercise 24	12
Exercise 25	13
Exercise 26	13
Exercise 27	13
Exercise 28	14
Exercise 29	14
Exercise 30	15
Exercise 31	15
Exercise 32	16
Exercise 33	16
Exercise 34	16
Exercise 35	19
Exercise 36	20

Chapter 1

Source note. Questions were extracted from the Chapter 1 problem section of *Modern Operating Systems*, 4th edition. Answers 1–33 were matched to the Chapter 1 answer section of the provided `MOS_4th_answer.pdf`. Exercises 34–36 had no matching answer in that answer section, so their answers are clearly labeled as prepared answers rather than answer-manual text. PDF page headers and footers were removed.

Exercise 1

Question

What are the two main functions of an operating system?

Answer

An operating system must provide the users with an extended machine, and it must manage the I/O devices and other system resources. To some extent, these are different functions.

Source: Answer manual.

Exercise 2

Question

In Section 1.4, nine different types of operating systems are described. Give a list of applications for each of these systems (one per operating systems type).

Answer

Obviously, there are a lot of possible answers. Here are some. Mainframe operating system: Claims processing in an insurance company. Server operating system: Speech-to-text conversion service for Siri. Multiprocessor operating system: Video editing and rendering. Personal computer operating system: Word processing application. Handheld computer operating system: Context-aware recommendation system. Embedded operating system: Programming a DVD recorder for recording TV. Sensor-node operating system: Monitoring temperature in a wilderness area. Real-time operating system: Air traffic control system. Smart-card operating system: Electronic payment.

Source: Answer manual.

Exercise 3

Question

What is the difference between timesharing and multiprogramming systems?

Answer

In a timesharing system, multiple users can access and perform computations on a computing system simultaneously using their own terminals. Multiprogramming systems allow a user to run multiple programs simultaneously. All timesharing systems are multiprogramming systems but not all multiprogramming systems are timesharing systems since a multiprogramming system may run on a PC with only one user.

Source: Answer manual.

Exercise 4

Question

To use cache memory, main memory is divided into cache lines, typically 32 or 64 bytes long. An entire cache line is cached at once. What is the advantage of caching an entire line instead of a single byte or word at a time?

Answer

Empirical evidence shows that memory access exhibits the principle of locality of reference, where if one location is read then the probability of accessing nearby locations next is very high, particularly the following memory locations. So, by caching an entire cache line, the probability of a cache hit next is increased. Also, modern hardware can do a block transfer of 32 or 64 bytes into a cache line much faster than reading the same data as individual words.

Source: Answer manual.

Exercise 5

Question

On early computers, every byte of data read or written was handled by the CPU (i.e., there was no DMA). What implications does this have for multiprogramming?

Answer

The prime reason for multiprogramming is to give the CPU something to do while waiting for I/O to complete. If there is no DMA, the CPU is fully occupied doing I/O, so there is nothing to be gained (at least in terms of CPU utilization) by multiprogramming. No matter how much I/O a program does, the CPU will be 100% busy. This of course assumes the major delay is the wait while data are copied. A CPU could do other work if the I/O were slow for other reasons (arriving on a serial line, for instance).

Source: Answer manual.

Exercise 6

Question

Instructions related to accessing I/O devices are typically privileged instructions, that is, they can be executed in kernel mode but not in user mode. Give a reason why these instructions are privileged.

Answer

Access to I/O devices (e.g., a printer) is typically restricted for different users. Some users may be allowed to print as many pages as they like, some users may not be allowed to print at all, while some users may be limited to printing only a certain number of pages. These restrictions are set by system administrators based on some policies. Such policies need to be enforced so that userlevel programs cannot interfere with them.

Source: Answer manual.

Exercise 7

Question

The family-of-computers idea was introduced in the 1960s with the IBM System/360 mainframes. Is this idea now dead as a doornail or does it live on?

Answer

It is still alive. For example, Intel makes Core i3, i5, and i7 CPUs with a variety of different properties including speed and power consumption. All of these machines are architecturally compatible. They differ only in price and performance, which is the essence of the family idea.

Source: Answer manual.

Exercise 8

Question

One reason GUIs were initially slow to be adopted was the cost of the hardware needed to support them. How much video RAM is needed to support a 25-line $\times$ 80-row character monochrome text screen? How much for a 1200 $\times$ 900-pixel 24-bit color bitmap? What was the cost of this RAM at 1980 prices (\$5/KB)? How much is it now?

Answer

A 25 $\times$ 80 character monochrome text screen requires a 2000-byte buffer. The 1200 $\times$ 900 pixel 24-bit color bitmap requires 3,240,000 bytes. In 1980 these two options would have cost \$10 and \$15,820, respectively. For current prices, check on how much RAM currently costs, probably pennies per MB.

Source: Answer manual.

Exercise 9

Question

There are several design goals in building an operating system, for example, resource utilization, timeliness, robustness, and so on. Give an example of two design goals that may contradict one another.

Answer

Consider fairness and real time. Fairness requires that each process be allocated its resources in a fair way, with no process getting more than its fair share. On the other hand, real time requires that resources be allocated based on the times when different processes must complete their execution. A realtime process may get a disproportionate share of the resources.

Source: Answer manual.

Exercise 10

Question

What is the difference between kernel and user mode? Explain how having two distinct modes aids in designing an operating system.

Answer

Most modern CPUs provide two modes of execution: kernel mode and user mode. The CPU can execute every instruction in its instruction set and use every feature of the hardware when executing in kernel mode. However, it can execute only a subset of instructions and use only subset of features when executing in the user mode. Having two modes allows designers to run user programs in user mode and thus deny them access to critical instructions.

Source: Answer manual.

Exercise 11

Question

A 255-GB disk has 65,536 cylinders with 255 sectors per track and 512 bytes per sector. How many platters and heads does this disk have? Assuming an average cylinder seek time of 11 ms, average rotational delay of 7 msec and reading rate of 100 MB/sec, calculate the average time it will take to read 400 KB from one sector.

Answer

Number of heads = $255 \text{ GB} / (65536 * 255 * 512) = 16$ Number of platters = $16/2 = 8$ The time for a read operation to complete is seek time + rotational latency + transfer time. The seek time is 11 ms, the rotational latency is 7 ms and the transfer time is 4 ms, so the average transfer takes 22 msec.

Source: Answer manual.

Exercise 12

Question

Which of the following instructions should be allowed only in kernel mode?

- (a) Disable all interrupts.
- (b) Read the time-of-day clock.
- (c) Set the time-of-day clock.
- (d) Change the memory map.

Answer

Choices (a), (c), and (d) should be restricted to kernel mode.

Source: Answer manual.

Exercise 13

Question

Consider a system that has two CPUs, each CPU having two threads (hyperthreading). Suppose three programs, P0, P1, and P2, are started with run times of 5, 10 and 20 msec, respectively. How long will it take to complete the execution of these programs? Assume that all three programs are 100% CPU bound, do not block during execution, and do not change CPUs once assigned.

Answer

It may take 20, 25 or 30 msec to complete the execution of these programs depending on how the operating system schedules them. If P0 and P1 are scheduled on the same CPU and P2 is scheduled on the other CPU, it will take 20 msec. If P0 and P2 are scheduled on the same CPU and P1 is scheduled on the other CPU, it will take 25 msec. If P1 and P2 are scheduled on the same CPU and P0 is scheduled on the other CPU, it will take 30 msec. If all three are on the same CPU, it will take 35 msec.

Source: Answer manual.

Exercise 14

Question

A computer has a pipeline with four stages. Each stage takes the same time to do its work, namely, 1 nsec. How many instructions per second can this machine execute?

Answer

Every nanosecond one instruction emerges from the pipeline. This means the machine is executing 1 billion instructions per second. It does not matter at all how many stages the pipeline has. A 10-stage pipeline with 1 nsec per stage would also execute 1 billion instructions per second. All that matters is how often a finished instruction pops out the end of the pipeline.

Source: Answer manual.

Exercise 15

Question

Consider a computer system that has cache memory, main memory (RAM) and disk, and an operating system that uses virtual memory. It takes 1 nsec to access a word from the cache, 10 nsec to access a word from the RAM, and 10 ms to access a word from the disk. If the cache hit rate is 95% and main memory hit rate (after a cache miss) is 99%, what is the average time to access a word?

Answer

Average access time = $0.95 \times 1 \text{ nsec}$ (word is in the cache) + $0.05 \times 0.99 \times 10 \text{ nsec}$ (word is in RAM, but not in the cache) + $0.05 \times 0.01 \times 10,000,000 \text{ nsec}$ (word on disk only) = 5001.445 nsec = 5.001445 μsec

Source: Answer manual.

Exercise 16

Question

When a user program makes a system call to read or write a disk file, it provides an indication of which file it wants, a pointer to the data buffer, and the count. Control is then transferred to the operating system, which calls the appropriate driver. Suppose that the driver starts the disk and terminates until an interrupt occurs. In the case of reading from the disk, obviously the caller will have to be blocked (because there are no data for it). What about the case of writing to the disk? Need the caller be blocked awaiting completion of the disk transfer?

Answer

Maybe. If the caller gets control back and immediately overwrites the data, when the write finally occurs, the wrong data will be written. However, if the driver first copies the data to a private buffer before returning, then the caller can be allowed to continue immediately. Another possibility is to allow the caller to continue and give it a signal when the buffer may be reused, but this is tricky and error prone.

Source: Answer manual.

Exercise 17

Question

What is a trap instruction? Explain its use in operating systems.

Answer

A trap instruction switches the execution mode of a CPU from the user mode to the kernel mode. This instruction allows a user program to invoke functions in the operating system kernel.

Source: Answer manual.

Exercise 18

Question

Why is the process table needed in a timesharing system? Is it also needed in personal computer systems running UNIX or Windows with a single user?

Answer

The process table is needed to store the state of a process that is currently suspended, either ready or blocked. Modern personal computer systems have dozens of processes running even when the user is doing nothing and no programs are open. They are checking for updates, loading email, and many other things. On a UNIX system, use the `ps -a` command to see them. On a Windows system, use the task manager.

Source: Answer manual.

Exercise 19

Question

Is there any reason why you might want to mount a file system on a nonempty directory? If so, what is it?

Answer

Mounting a file system makes any files already in the mount-point directory inaccessible, so mount points are normally empty. However, a system administrator might want to copy some of the most important files normally located in the mounted directory to the mount point so they could be found in their normal path in an emergency when the mounted device

was being repaired.

Source: Answer manual.

Exercise 20

Question

For each of the following system calls, give a condition that causes it to fail: fork, exec, and unlink.

Answer

Fork can fail if there are no free slots left in the process table (and possibly if there is no memory or swap space left). Exec can fail if the file name given does not exist or is not a valid executable file. Unlink can fail if the file to be unlinked does not exist or the calling process does not have the authority to unlink it.

Source: Answer manual.

Exercise 21

Question

What type of multiplexing (time, space, or both) can be used for sharing the following resources: CPU, memory, disk, network card, printer, keyboard, and display?

Answer

Time multiplexing: CPU, network card, printer, keyboard. Space multiplexing: memory, disk. Both: display.

Source: Answer manual.

Exercise 22

Question

Can the `count = write(fd, buffer, nbytes);` call return any value in count other than nbytes? If so, why?

Answer

If the call fails, for example because `fd` is incorrect, it can return `-1`. It can also fail because the disk is full and it is not possible to write the number of bytes requested. On a correct termination, it always returns `nbytes`.

Source: Answer manual.

Exercise 23

Question

A file whose file descriptor is `fd` contains the following sequence of bytes: 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5. The following system calls are made: `lseek(fd, 3, SEEK SET)`; `read(fd, &buffer, 4)`; where the `lseek` call makes a seek to byte 3 of the file. What does `buffer` contain after the read has completed?

Answer

It contains the bytes: 1, 5, 9, 2.

Source: Answer manual.

Exercise 24

Question

Suppose that a 10-MB file is stored on a disk on the same track (track 50) in consecutive sectors. The disk arm is currently situated over track number 100. How long will it take to retrieve this file from the disk? Assume that it takes about 1 ms to move the arm from one cylinder to the next and about 5 ms for the sector where the beginning of the file is stored to rotate under the head. Also, assume that reading occurs at a rate of 200 MB/s.

Answer

Time to retrieve the file = $1 * 50$ ms (Time to move the arm over track 50) + 5 ms (Time for the first sector to rotate under the head) + $10/200 * 1000$ ms (Read 10 MB) = 105 ms

Source: Answer manual.

Exercise 25

Question

What is the essential difference between a block special file and a character special file?

Answer

Block special files consist of numbered blocks, each of which can be read or written independently of all the other ones. It is possible to seek to any block and start reading or writing. This is not possible with character special files.

Source: Answer manual.

Exercise 26

Question

In the example given in Fig. 1-17, the library procedure is called read and the system call itself is called read. Is it essential that both of these have the same name? If not, which one is more important?

Answer

System calls do not really have names, other than in a documentation sense. When the library procedure read traps to the kernel, it puts the number of the system call in a register or on the stack. This number is used to index into a table. There is really no name used anywhere. On the other hand, the name of the library procedure is very important, since that is what appears in the program.

Source: Answer manual.

Exercise 27

Question

Modern operating systems decouple a process address space from the machine's physical memory. List two advantages of this design.

Answer

This allows an executable program to be loaded in different parts of the machine's memory in different runs. Also, it enables program size to exceed the size of the machine's memory.

Source: Answer manual.

Exercise 28

Question

To a programmer, a system call looks like any other call to a library procedure. Is it important that a programmer know which library procedures result in system calls? Under what circumstances and why?

Answer

As far as program logic is concerned, it does not matter whether a call to a library procedure results in a system call. But if performance is an issue, if a task can be accomplished without a system call the program will run faster. Every system call involves overhead time in switching from the user context to the kernel context. Furthermore, on a multiuser system the operating system may schedule another process to run when a system call completes, further slowing the progress in real time of a calling process.

Source: Answer manual.

Exercise 29

Question

Figure 1-23 shows that a number of UNIX system calls have no Win32 API equivalents. For each of the calls listed as having no Win32 equivalent, what are the consequences for a programmer of converting a UNIX program to run under Windows?

Answer

Several UNIX calls have no counterpart in the Win32 API: Link: a Win32 program cannot refer to a file by an alternative name or see it in more than one directory. Also, attempting to create a link is a convenient way to test for and create a lock on a file. Mount and umount: a Windows program cannot make assumptions about standard path names because on systems with multiple disk drives the drive-name part of the path may be different. Chmod: Windows uses access control lists. Kill: Windows programmers cannot kill a misbehaving program that is not cooperating.

Source: Answer manual.

Exercise 30

Question

A portable operating system is one that can be ported from one system architecture to another without any modification. Explain why it is infeasible to build an operating system that is completely portable. Describe two high-level layers that you will have in designing an operating system that is highly portable.

Answer

Every system architecture has its own set of instructions that it can execute. Thus a Pentium cannot execute SPARC programs and a SPARC cannot execute Pentium programs. Also, different architectures differ in bus architecture used (such as VME, ISA, PCI, MCA, SBus, ...) as well as the word size of the CPU (usually 32 or 64 bit). Because of these differences in hardware, it is not feasible to build an operating system that is completely portable. A highly portable operating system will consist of two high-level layers—a machine-dependent layer and a machine-independent layer. The machine-dependent layer addresses the specifics of the hardware and must be implemented separately for every architecture. This layer provides a uniform interface on which the machine-independent layer is built. The machine-independent layer has to be implemented only once. To be highly portable, the size of the machine-dependent layer must be kept as small as possible.

Source: Answer manual.

Exercise 31

Question

Explain how separation of policy and mechanism aids in building microkernel-based operating systems.

Answer

Separation of policy and mechanism allows OS designers to implement a small number of basic primitives in the kernel. These primitives are simplified, because they are not dependent of any specific policy. They can then be used to implement more complex mechanisms and policies at the user level.

Source: Answer manual.

Exercise 32

Question

Virtual machines have become very popular for a variety of reasons. Nevertheless, they have some downsides. Name one.

Answer

The virtualization layer introduces increased memory usage and processor overhead as well as increased performance overhead.

Source: Answer manual.

Exercise 33

Question

Here are some questions for practicing unit conversions:

- (a) How long is a nanoyear in seconds?
- (b) Micrometers are often called microns. How long is a megam micron?
- (c) How many bytes are there in a 1-PB memory?
- (d) The mass of the earth is 6000 yottagrams. What is that in kilograms?

Answer

The conversions are straightforward:

- (a) A nanoyear is $10^{-9} \times 365 \times 24 \times 3600 = 31.536$ msec.
- (b) 1 meter
- (c) There are 2^{50} bytes, which is 1,099,511,627,776 bytes.
- (d) It is 6×10^{24} kg or 6×10^{27} g.

Source: Answer manual.

Exercise 34

Question

Write a shell that is similar to Fig. 1-19 but contains enough code that it actually works so you can test it. You might also add some features such as redirection of input and output, pipes, and background jobs.

Answer

One working answer is a small POSIX-style shell that repeatedly reads a command line, creates a child process with `fork()`, runs the requested program in the child with `execvp()`, and lets the parent wait with `waitpid()` unless the command ends with `&`. This follows the Chapter 1 shell example and process-management system-call model.

Save this as `tinysh.c`:

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#define MAXLINE 1024
#define MAXARGS 64

static int parse(char *line, char *argv[]) {
    int argc = 0;
    char *tok = strtok(line, " \\t\\n");
    while (tok != NULL && argc < MAXARGS - 1) {
        argv[argc++] = tok;
        tok = strtok(NULL, " \\t\\n");
    }
    argv[argc] = NULL;
    return argc;
}

int main(void) {
    char line[MAXLINE];
    char *argv[MAXARGS];

    for (;;) {
        printf("tinysh$ ");
        fflush(stdout);

        if (fgets(line, sizeof line, stdin) == NULL) {
            putchar('\\n');
            break;
        }

        int argc = parse(line, argv);
        if (argc == 0) continue;
```

```

    if (strcmp(argv[0], "exit") == 0) break;

    int background = 0;
    if (strcmp(argv[argc - 1], "&") == 0) {
        background = 1;
        argv[-argc] = NULL;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        continue;
    }

    if (pid == 0) {
        execvp(argv[0], argv);
        perror(argv[0]);
        _exit(127);
    }

    if (!background) {
        int status;
        while (waitpid(pid, &status, 0) < 0 && errno == EINTR) {
            /* retry if interrupted */
        }
        else {
            printf("[background pid %ld]\n", (long) pid);
        }
    }

    return 0;
}

```

Compile and run it on a UNIX-like system:

```

cc -Wall -Wextra -O2 tinysh.c -o tinysh
./tinysh
tinysh$ pwd
tinysh$ ls
tinysh$ sleep 10 &
tinysh$ exit

```

Expected behavior: foreground commands run and return to the prompt after completion; a command followed by `&` starts in the background and immediately returns to the prompt. If a program name is invalid, the child prints an `execvp()` error.

Limitations: this minimal shell does not implement quoting, pipes, or redirection. Those can be added using the same Chapter 1 ideas: parse `<` and `>` and apply `dup2()` before `execvp()`, and parse `|` using `pipe()`, multiple children, and descriptor closing.

Safety: run this in an ordinary user account, not as administrator/root. Test only harmless commands first. A shell bug can leave background processes running, so use `jobs`, `ps`, or `kill` from another terminal if needed.

Source: Prepared from textbook concepts.

Exercise 35

Question

If you have a personal UNIX-like system (Linux, MINIX 3, FreeBSD, etc.) available that you can safely crash and reboot, write a shell script that attempts to create an unlimited number of child processes and observe what happens. Before running the experiment, type `sync` to the shell to flush the file system buffers to disk to avoid ruining the file system. You can also do the experiment safely in a virtual machine. Note: Do not try this on a shared system without first getting permission from the system administrator. The consequences will be instantly obvious so you are likely to be caught and sanctions may follow.

Answer

This is a dangerous resource-exhaustion experiment. It should not be run on a shared machine or on a host that contains important unsaved work. I did not execute this experiment. The unsafe idea is an unbounded loop that repeatedly starts child processes. A safer version is to impose a process limit before running the test and to use a bounded loop:

```
# fork-test.sh
#!/bin/sh
MAX_CHILDREN=${MAX_CHILDREN:-100}
SLEEP_SECONDS=${SLEEP_SECONDS:-300}

i=0
while [ "$i" -lt "$MAX_CHILDREN" ]; do
    (sleep "$SLEEP_SECONDS") &
    i=$((i + 1))
done

echo "Started $i child processes. Parent shell PID is $$."
echo "Use: pkill -P $$"
wait
```

Safer test commands inside a disposable virtual machine or container:

```
chmod +x fork-test.sh
ulimit -u 128
MAX_CHILDREN=100 ./fork-test.sh
```

Expected behavior: the system will allow children to be created until either the script reaches `MAX_CHILDREN` or a per-user/process limit is reached. Once the limit is reached, later `fork()` attempts normally fail and shells or programs may print messages such as **Resource temporarily unavailable**. Interactive login, opening new terminals, and starting editors may become slow or temporarily impossible if the limit is too high.

Important limits involved: UNIX-like systems commonly enforce a per-user process limit such as `RLIMIT_NPROC`, visible or set by `ulimit -u`. Linux systems may also apply cgroup PID limits such as `pids.max`; service managers may apply task limits; the kernel has a maximum PID range; and total memory can become the practical limit before PID space is exhausted. Recovery steps: first try `pkill -P <parent-pid>` to kill the children of the test shell. If that is not enough, use another terminal or VM console and run `pkill -u <your-user> sleep` only when you are sure the test children are `sleep` processes. If the session is unusable, reboot the disposable VM. Before deliberately stressing process creation, run `sync` as the question says, keep the test inside a VM, and avoid shared systems.

Source: Independently prepared technical answer.

Exercise 36

Question

Examine and try to interpret the contents of a UNIX-like or Windows directory with a tool like the UNIX `od` program. (Hint: How you do this will depend upon what the OS allows. One trick that may work is to create a directory on a USB stick with one operating system and then read the raw device data using a different operating system that allows such access.)

Answer

A directory is stored by the file system as structured metadata mapping file names to inode numbers or file identifiers. The exact byte layout is file-system-specific, so the safest experiment is to inspect a disposable file-system image rather than a real system directory. I did not execute this experiment.

On some UNIX-like systems, a directory may be readable as bytes, but many modern systems reject direct reads of directories. A quick non-destructive check is:

```
mkdir dirtest
touch dirtest/alpha dirtest/beta
ls -lai dirtest
od -Ax -tx1z dirtest
```

Expected behavior: `ls -lai` should show inode numbers and names such as `.`, `..`, `alpha`, and `beta`. The `od` command may either dump directory bytes or fail with an error such as `Is a directory`; both outcomes are normal depending on the operating system and file system. A safer raw-layout experiment is to use a file-system image in a VM:

```
dd if=/dev/zero of=fs.img bs=1M count=16
mkfs.ext2 -F fs.img
mkdir mnt
sudo mount -o loop fs.img mnt
sudo mkdir mnt/demo
sudo touch mnt/demo/alpha mnt/demo/beta
sync
sudo umount mnt
debugfs -R "ls -l /demo" fs.img
debugfs -R "stat /demo" fs.img
```

Expected behavior: `debugfs` should show directory entries and metadata for `/demo`. The names should correspond to the files created, while the numeric identifiers reflect the file system's internal allocation. If a raw directory block is dumped, the bytes will not be plain text only; they encode record lengths, inode numbers, file types, and names according to that file system's format.

Safety warnings: do not run raw-device commands on your main disk. Use a disposable image file, USB stick, container, or virtual machine. If using `sudo mount`, always unmount with `sudo umount mnt` before deleting the image. Avoid writing to raw devices unless you are certain of the target path.

Source: Independently prepared technical answer.